Course: MMDS

Tutorial 01 — Set up Hadoop (Fully Distributed, Docker)

Hadoop 3.2.1 · 5 Nodes (1 NN + 1 2NN + 3 DN) · Docker on Windows

---

1. Prerequisites

Ensure the following are installed on your Windows machine before proceeding:

- Docker Desktop for Windows (WSL 2 backend recommended)
- Git Bash or PowerShell (to run commands below)
- At least 8 GB RAM and 20 GB free disk space

> 💡 *Enable 'Use the WSL 2 based engine' in Docker Desktop → Settings → General for best performance on Windows.*

2. Project File Structure

Create a folder called hadoop-docker/ with the following structure:

```
hadoop-docker/
├── Dockerfile
├── docker-compose.yml
├── entrypoint.sh
└── config/
    ├── core-site.xml
    ├── hdfs-site.xml
    ├── mapred-site.xml
    ├── yarn-site.xml
    └── workers
```

File	Purpose
`Dockerfile`	Builds the shared Hadoop base image (Java 8, SSH, Hadoop 3.2.1)
`docker-compose.yml`	Defines all 5 nodes, volumes, networks, and port mappings
`entrypoint.sh`	Start script: launches NameNode, 2NN, DataNode, YARN per NODE_TYPE env var
`config/core-site.xml`	Sets HDFS default FS to hdfs://namenode:9000
`config/hdfs-site.xml`	Replication factor = 3; name/data dirs; secondary NN address
`config/mapred-site.xml`	MapReduce framework = YARN; job history server address

config/yarn-site.xml	ResourceManager on namenode:8032; NodeManager shuffle service
config/workers	Lists datanode1, datanode2, datanode3

3. Cluster Architecture

The cluster runs 5 Docker containers connected on a shared bridge network (hadoop_net):

Container	Role	Exposed Ports
namenode	NameNode	9870 (HDFS UI), 9000 (IPC), 8088 (YARN UI), 19888 (History)
secondarynamenode	Secondary NameNode	9868 (2NN UI)
datanode1	DataNode 1	9864 (DN UI), 8042 (NM UI)
datanode2	DataNode 2	9865→9864, 8043→8042
datanode3	DataNode 3	9866→9864, 8044→8042

> 💡 *All containers share the same Docker image built from the Dockerfile. The NODE_TYPE environment variable controls which daemons are started at runtime.*

4. Configuration Files

4.1 Dockerfile

The Dockerfile builds the common Hadoop base image shared by all 5 nodes:

```
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive

# Install Java 8, SSH, pdsh
RUN apt-get update && apt-get install -y \
    openjdk-8-jdk ssh pdsh wget && \
    rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
ENV PATH=$JAVA_HOME/bin:$PATH

# Download Hadoop 3.2.1
RUN wget -q https://archive.apache.org/dist/hadoop/common/\
    hadoop-3.2.1/hadoop-3.2.1.tar.gz -P /tmp && \
    tar -xzf /tmp/hadoop-3.2.1.tar.gz -C /opt/ && \
    mv /opt/hadoop-3.2.1 /opt/hadoop

ENV HADOOP_HOME=/opt/hadoop
ENV PATH=$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH

# Setup passwordless SSH
RUN ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa && \
    cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys && \
```

```
    chmod 0600 ~/.ssh/authorized_keys && \
    echo ssh > /etc/pdsh/rcmd_default

COPY config/ $HADOOP_HOME/etc/hadoop/
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

4.2 core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:9000</value>
  </property>
</configuration>
```

4.3 hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name><value>3</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/hadoop/dfs/name</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/hadoop/dfs/data</value>
  </property>
  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>secondarynamenode:9868</value>
  </property>
</configuration>
```

4.4 mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:
    $HADOOP_MAPRED_HOME/share/hadoop/mapreduce/lib/*</value>
  </property>
</configuration>
```

4.5 yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>namenode</value>
  </property>
</configuration>
```

5. Building and Starting the Cluster

Open a terminal (Git Bash / PowerShell) in the hadoop-docker/ folder and run:

Step 1 — Build the Docker image

```
docker compose build
```

> 💡 *This downloads Ubuntu 20.04, installs Java 8, and downloads Hadoop 3.2.1 (~500 MB). It only runs once; subsequent builds use the cache.*

Step 2 — Start all 5 nodes

```
docker compose up -d
```

Wait ~30 seconds for all daemons to initialize. Check container status:

```
docker compose ps
```

Expected output: all 5 containers show State = Up.

Step 3 — View container logs (optional)

```
docker logs namenode
docker logs datanode1
```

6. Verifying the Installation

6.1 Web UIs

Open the following URLs in your browser:

- HDFS NameNode: http://localhost:9870 (Overview, Datanodes, Utilities)
- YARN ResourceManager: http://localhost:8088 (Cluster nodes, applications)
- Secondary NameNode: http://localhost:9868

> 💡 *On the HDFS Overview page you should see 3 Live Nodes and a replication factor of 3.*

6.2 Check DataNodes via CLI

Run the following command inside the namenode container:

```
docker exec -it namenode bash
hdfs dfsadmin -report
```

This reports all 3 DataNodes as Live. Exit with Ctrl+D.

7. Running the Word Count Job

Execute the following commands inside the namenode container to run the provided Word Count MapReduce example and verify the cluster:

```
docker exec -it namenode bash
```

1. Create the input directory in HDFS:
   ```
   hdfs dfs -mkdir -p /user/root/input
   ```
2. Upload Hadoop XML config files as input:
   ```
   hdfs dfs -put $HADOOP_HOME/etc/hadoop/*.xml /user/root/input/
   ```
3. Run the Word Count job:
   ```
   hadoop jar \
   $HADOOP_HOME/share/hadoop/mapreduce/\
   hadoop-mapreduce-examples-3.2.1.jar \
   wordcount /user/root/input /user/root/output
   ```
4. View the output:
   ```
   hdfs dfs -cat /user/root/output/part-r-00000 | head -20
   ```

> 💡 *The job should complete successfully, showing map 100% reduce 100% in the log. The output lists each word from the XML files and its count.*

To track job progress, open http://localhost:8088 and navigate to Applications → Running while the job executes.

8. Stopping the Cluster

To stop all containers while preserving HDFS data (Docker volumes):

```
docker compose down
```

To stop and completely remove all data volumes (full reset):

```
docker compose down -v
```

9. Troubleshooting

- DataNode not showing in dfsadmin -report: wait 20 s and retry; the DataNode needs time to register with the NameNode.
- Port already in use: ensure nothing else binds to 9870, 9000, 8088 on your Windows host. Check with: netstat -ano | findstr :9870
- NameNode format loop on restart: the entrypoint only formats if /hadoop/dfs/name/current does not exist, so Docker volumes prevent re-formatting.
- Docker Desktop WSL 2 out of memory: increase Docker Desktop memory limit to at least 6 GB in Settings → Resources.

10. References

- Apache Hadoop 3.2.1 Single Cluster Guide: https://hadoop.apache.org/docs/r3.2.1/hadoop-project-dist/hadoop-common/SingleCluster.html
- Docker Desktop WSL 2 backend: https://docs.docker.com/desktop/windows/wsl/
- Hadoop Fully Distributed Mode: https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/ClusterSetup.html